German University in Cairo GUC
Faculty of Media Engineering and Technology
Prof. Dr. Mohammed Abdel-Megeed Salem
Eng. Dinah Waref Shalaby

CSEN1121—Computational Intelligence and Neural Networks
Winter Semester 2026

CINN1121 Project

Task 1 due Friday 6/11/2026

Task 2 due Thursday 15/11/2026

Tasks 3, 4, and 5 due on Sunday 29 November 2026

Total 150 Points

Setup and regulations

1. Work in teams is allowed for up to 5 students. One student should submit the project in behalf of the team. Submission of the team is due on **Friday 23/10/2026** to the following form: https://forms.gle/NqZzgFCZigBviThU8
2. Put all files in a .zip file and name it: CINN_WS2026_Project_[Teamname].zip. Submit file to: https://forms.gle/34m3J59vaR6pW4Jb7
3. Add a project report. It should describe the operations and the results. Any notes to be considered in the grading should be added in this report too.
4. Add full details about the course, the project, and the team on the front page of the project report.
5. Use Jupyter Notebook/Colab to test your work. Name the file exactly the same as the .zip file.
6. Every task should be collected in ONE single cell block, starting with this comment format:

   #Task number

   #Task requirement

   #Task overview
7. The code should be commented to explain the steps you wrote.
8. Any AI-generated or plagiarized submission will be graded as zero.

German University in Cairo GUC
Faculty of Media Engineering and Technology
Prof. Dr. Mohammed Abdel-Megeed Salem
Eng. Dinah Waref Shalaby

CSEN1121—Computational Intelligence and Neural Networks
Winter Semester 2026

Task 1 — Dataset, Preprocessing, and Splitting (10 Marks) Due Friday 6/11/2026

1. Choose a small color-image dataset (you should choose a classification problem and try to solve it (e.g., scenes, animals, plants,..). Keep images small (e.g., 16×16 or 32×32) or work on small patches a from-scratch network trains slowly, so image size should stay computationally reasonable.
2. For every image, generate the grayscale input (the L / lightness channel, or a standard RGB→gray conversion) and the color target (RGB, or the a/b chrominance channels if you work in Lab space).
3. Normalize all pixel values (e.g., to the [0, 1] range).
4. Visualize sample grayscale/color pairs and plot pixel-value distributions to sanity-check the data.
5. Split the data into Training (70%), Validation (20%), and Testing (10%) sets.
6. Divide the training data into batches of 100 samples each.

Task 2 — Neural Network Architecture (Built From Scratch) (15 Marks) Due Thursday 15/11/2026

1. Implement your own network class or set of matrix operations — do not import a pre-built layer or model class.
2. Input layer size = number of grayscale features you feed in (e.g., flattened pixel values of an image or patch).
3. Output layer size = number of color values you predict per image/patch (e.g., 2 for a/b channels, or 3 for RGB, per pixel).
4. Choose the number of hidden layers and neurons, and implement the activation functions yourself (e.g., ReLU for hidden layers, sigmoid or tanh for the output layer, matched to your normalized color range).
5. Initialize weights and biases with small random values (implement the initialization yourself).

Task 3 — Backpropagation, Implemented From Scratch (60 Marks) Due on Sunday 29 November 2026

This is the core of the project and carries the most weight. Everything below must be your own code operating on plain arrays/matrices (numpy is fine for the arithmetic; it must not provide the derivatives or the update rule for you).

1. Forward pass: compute each layer's weighted sum and activation output, ending in the predicted color values.
2. Loss: compute mean squared error between predicted and true color values.

3. Backward pass: derive and implement the gradient of the loss with respect to every weight and bias, propagating the error from the output layer back to the input layer using the chain rule.
4. Weight update: apply gradient descent with a learning rate you choose (e.g., start around 0.1 and tune from there).
5. Training loop: for each epoch, iterate over all batches, running one forward + backward pass and one weight update per batch; run validation once at the end of every epoch; run the test pass exactly once, after all training is complete.
6. Train for at least 10 epochs.

Task 4 — Evaluation and Results (25 Marks) Due on Sunday 29 November 2026

1. Plot training and validation error against epoch number.
2. Report the final test error.
3. For at least 5–10 test images, show the grayscale input, your network's colorized output, and the true color image side by side.
4. Discuss what you observed: which hyperparameters (learning rate, epochs, architecture, batch size) mattered most, and any limitations (e.g., outputs looking desaturated or averaged, which is a known effect of training a colorizer with plain MSE loss).

Task 5 — Genetic Algorithm Comparison (20 Marks) Due on Sunday 29 November 2026

Keep the same network architecture from Task 2, but optimize its weights with a genetic algorithm instead of backpropagation: represent each individual as a full set of weights, evaluate fitness with the same error function as Task 3, and implement selection, crossover, mutation, and replacement. Compare the two approaches on convergence speed, final error, and stability, and include this comparison in your report.